

FENRIR: A Fast-Weight Mixer for Single-Pass Chain Composition

Voidwolf¹

¹*Independent researcher, research@voidwolf.space*

Abstract

We study when and how a fast-weight mixer with a state-dependent write address learns to compose 2-hop retrieval chains inside a single forward pass. The mixer, which we call FENRIR, computes an eager lookup $a_t = M_{t-1}k_t$ into the current state, adds it to the current key to form $k_{\text{eff}} = k_t + a_t$, and writes the outer product $\beta_t k_{\text{eff}} v_t^\top$ into the fast-weight state. On a synthetic 2-hop chain-composition task at $K_1 \in \{1, \dots, 4\}$ with $K_2 = 4$ candidate values, we train up to $n = 10$ seeds per variant under matched protocol and report four mechanism-level findings: (a) chain composition to $K_1 = 4$ inside a single forward pass is attainable, but formation is bimodal: under mixed-mode training 5/10 rev seeds cross the ladder to $\text{acc} \geq 0.895$ while the other 5/10 stall at chance; under joint-mode training 6/10 pass at $\text{acc} \geq 0.958$; the fwd operator direction has a much lower per-seed pass probability (0/5 mixed; 1/5 joint at 0.977) but is not categorically failing under joint mode; (b) on passing seeds the answer becomes decodable from the third layer’s residual at accuracy matching training, while failing seeds show no localisation at any layer; (c) an inference-time knockout of the eager address-perturbation on trained checkpoints collapses passing-seed accuracy from ~ 0.96 to ~ 0.03 (worse than chance), with fwd checkpoints collapsing to 0.000, showing the term is not incidental but the computational pathway the model routes through; (d) a mid-scan probe traces this pathway to a buildup inside the state during the hop-2 write window, and zeroing the eager term at probe time collapses the buildup consistent with (c). We compare two operator directions (reverse-lookup $k + Mk$ vs forward-lookup $k + M^\top k$) and report their empirical envelopes side by side. We make no strong claim of primitive-level novelty; the paper’s contribution is the mechanistic characterisation and the audit-safe reproduction workflow. All code, tests, and mechanism probes are packaged as a standalone repository, and every number in the paper is traceable to a committed JSON.

1 Introduction

A *chain composition* step follows two facts through their shared intermediate: given (a, b) and (b, c) known, retrieve c from a . Standard fast-weight and delta-family mixers store each fact independently and depend on stacking multiple layers, or multiple forward passes, to compose across them. The mechanism question we ask in this paper is narrower: given a mixer whose write address itself depends on the current state, when and how does it learn to compose a 2-hop chain inside a single forward pass?

The primitive we study, which we call FENRIR, is an address-perturbed fast-weight mixer with two operator directions we treat as sibling variants: *reverse-lookup* (rev) with $k_{\text{eff}} = k + Mk$ and *forward-lookup* (fwd) with $k_{\text{eff}} = k + M^\top k$. Both share the same outer-product write and the same forward read; both belong to the outer-product / delta-family lineage (§7). We do not claim the primitive itself is new. Prior fast-weight work has explored many state-dependent write variants and it is plausible that the FENRIR update rule appears somewhere under a different name; we hedge this in §7 and give the closest prior art we found.

What the paper does claim is a mechanism-level account of when composition forms in this primitive and what it looks like inside the model when it does. Concretely:

1. **Bimodal formation (§5.2).** Under a matched-protocol $n = 5$ -seed run at $K_1 = 4$, under mixed-mode

training 5/10 rev seeds cross the full K1 ladder to $\text{acc} \geq 0.895$; under joint-mode training 6/10 pass at $\text{acc} \geq 0.958$. The failing cluster in both curricula lands at chance; passing and failing seeds sit at cleanly separated clusters. The fwd operator direction has a much lower per-seed pass probability (0/5 mixed; 1/5 joint at 0.977), showing curriculum can partially rescue fwd formation. A single-seed report would report one number from that distribution and be misleading.

2. **Layer localisation (§5.3).** On passing seeds a logit-lens probe shows the answer becoming decodable at the third layer’s residual output at accuracy matching training. On failing seeds no layer localises anything. The distinction between passing and failing is not distributed; it is a specific layer’s behaviour.
3. **Inference-time causal test (§6.1).** On trained checkpoints, forcing the mixer’s eager term to zero at every position of the forward pass collapses passing-rev accuracy from $\text{acc} \sim 0.96$ to $\text{acc} \sim 0.03$ (worse than chance), and collapses fwd checkpoints to 0.000 hits. The eager address-perturbation is not incidental; it is the computational pathway the model routes through.
4. **Mid-scan mechanism (§5.4).** A per-position probe of the mixer state at the layer where composition is constructed shows the answer signal building up inside the state during the hop-2 write window; zeroing the eager term at probe time collapses the buildup, consistent with the inference-time result above.
5. **Audit workflow (§4.5).** The mechanism probes read from the mixer’s own cache or use the mixer’s own eager operator method, so an external reconstruction cannot silently compute a different operator than the mixer used. Every convention claim (M storage direction, eager operator per variant, read direction) is expressed as a pytest test; the repository is a working example of an audit workflow that catches the operator-convention bug class described in Appendix B.

The paper’s tables read from JSON files committed under `code/outputs/`. Every number in the text is traceable to one. Given the formation-mode bimodality above, the reader should treat all per-seed numbers as the raw ingredient, and any per-variant summary statistic as an aggregate over that distribution rather than a representative point estimate.

2 Background

Fast-weight programmers. A linear-attention layer can be rewritten as a fast-weight programmer whose associative memory $M \in \mathbb{R}^{d \times d}$ is updated by outer-product writes and read by matrix-vector products (Schmidhuber, 1992; Schlag et al., 2021). Each per-position write $M += k_t v_t^\top$ stores an association between a projected key and value; each read $y_t = M^\top q_t$ retrieves values by matching the query against stored keys.

Delta-family updates. A pure additive write accumulates without bound and interferes as the state fills. The delta rule fixes this by writing a residual against the current prediction: $M += \beta_t k_t (v_t - M^\top k_t)^\top$ (Irie et al., 2022b). Yang et al. (2024b) showed the delta rule admits a chunk-parallel form via a triangular closure, making delta-family mixers tractable at LM scale; Yang et al. (2024a) added scalar gating.

Chain composition and associative recall. Small-scale compositional-recall tasks are a common substrate for probing mixer capabilities. Zoology and BASED (Arora et al., 2023, 2024) study single-hop associative recall in linear attention and gated recurrent architectures. Grokked-Transformers (Wang et al., 2024) shows dense transformers acquire multi-hop compositional recall only under extended training, framing chain composition as a capability gap worth closing with better architectural inductive biases.

Mechanism probes. Logit-lens (nostalgebraist, 2020) reads intermediate residuals through the model’s own LM head to locate where a target token becomes decodable. Induction-head style circuit work (Olsson et al., 2022) makes causal claims about intermediate representations via targeted knockouts. Our mid-scan probe extends this pattern to per-timestep reads of the fast-weight state.

State-space scaffolds. The FENRIR block wraps its mixer in a Mamba (Gu and Dao, 2023; Dao and Gu, 2024) S4-style scaffold (in_proj, causal depthwise conv, z-gate, output projection), so the comparison against Mamba as a non-fast-weight baseline is a swap-in change to the mixer only.

3 Method

3.1 The FENRIR mixer

At each position t , given projections $q_t, k_t, v_t \in \mathbb{R}^d$ and a scalar gate $\beta_t \in (0, 1)$, the mixer maintains a fast-weight state $M \in \mathbb{R}^{d \times d}$ under the storage convention $M[i, j] = k_{\text{eff}}[i] \cdot v[j]$ (keys index axis 0, values index axis 1). The per-position update is

$$a_t = \begin{cases} M_{t-1} k_t & \text{(rev variant)} \\ M_{t-1}^\top k_t & \text{(fwd variant)} \end{cases} \quad (1)$$

$$k_{\text{eff},t} = k_t + a_t \quad (2)$$

$$M_t = M_{t-1} + \beta_t k_{\text{eff},t} v_t^\top \quad (3)$$

$$y_t = M_t^\top q_t \quad (4)$$

The two variants differ only in the direction of the eager address-perturbation a_t : **rev** matches k_t against stored values and returns stored keys; **fwd** matches k_t against stored keys and returns stored values. The write and the read are identical across variants.

Both variants require $d_{\text{key}} = d_{\text{value}}$ so $k_{\text{eff},t}$ type-checks. The mixer’s forward pass uses a sequential per-position loop. For **rev** we additionally provide a chunk-parallel kernel that reduces each chunk to a single unit-lower- triangular solve; the derivation is standard and appears in the appendix.

3.2 Scaffold

Each FENRIR block wraps the mixer in a Mamba-style S4 scaffold: in_proj $\rightarrow$ depthwise causal conv1d $\rightarrow$ SiLU $\rightarrow$ z-gate $\rightarrow$ mixer $\rightarrow$ readout $\rightarrow$ elementwise gate by SiLU(z) $\rightarrow$ out_proj. Blocks are stacked with pre-norm RMS residuals; the final residual reads out through a tied LM head.

4 Experimental setup

4.1 Task

We use a 2-hop injective chain-composition task with a *truncated*-answer supervision protocol. Each episode has:

- K_1 hop-1 facts of the form (e_i, b_i) (entity $\rightarrow$ bridge);
- K_2 hop-2 facts of the form (c_j, v_j) (bridge-key $\rightarrow$ value); the bridges $\{b_i\}$ are an *injective* subset of $\{c_j\}$ enforced by **randperm**;
- a query $q = e_{\text{target}}$;
- the answer $v_{f(\text{target})}$ where $f : [K_1] \rightarrow [K_2]$ is the injective map $b_i = c_{f(i)}$.

The sequence layout is FACT $e_1 b_1$ SEP $e_2 b_2$ SEP ... SEP $c_1 v_1$ SEP $c_2 v_2$... QTOK q ATOK and the model is teacher-forced up to and including ATOK; loss is cross-entropy against the answer token at the ATOK position. Chance is $1/K_2 = 0.25$ for $K_2 = 4$. The injective constraint blocks a “collision luck” shortcut that inflates chain accuracy on the non-injective version of this task.

4.2 Model configuration

Unless noted, $d_{\text{model}} = 256$, $d_{\text{key}} = d_{\text{value}} = 32$, 3 FENRIR blocks, `expand = 2`, conv kernel = 4. Vocabulary is 263 tokens (7 control tokens + 128 entities + 128 values). Parameter count is 1,453,827.

4.3 Training protocol

Optimizer is AdamW with $\text{lr} = 3\text{e-}4$, $\text{betas} = (0.9, 0.95)$, $\text{weight_decay} = 0.01$. LR schedule is cosine with 200-step linear warmup and floor $3\text{e-}5$. Precision is bf16 autocast on CUDA. Gradient clip is 1.0 (global norm). Batch size = 64. Two training curricula are compared:

- **mixed** (Table 1): at each step, cycle through $K_1 \in \{1, \dots, 4\}$. 4000 steps total.
- **joint** (Table 2): at each step, sample one batch each at $K_1 = 1$ and $K_1 = 4$, average the cross-entropy losses. 5000 steps total.

Optimizer hyperparameters follow the large-scale LM convention (GPT-3, Chinchilla, LLaMA) rather than PyTorch’s Adam defaults ($\beta_2 = 0.95$ rather than $\beta_2 = 0.999$). At this experiment’s scale ($\sim 1.5\text{M}$ parameters, $\lesssim 5000$ steps) the choice has limited effect. We did not ablate it.

4.4 Reference hardware

All numbers reported here were produced on an NVIDIA RTX 5090 (32 GB, CUDA 13.3, PyTorch 2.x, pinned as `torch>=2.1` in the reproduction repository’s `requirements.txt`) under bf16 autocast.

4.5 Audit as code

Every claim about the mixer’s storage convention or its eager operator per variant is expressed as a pytest test in `code/tests/test_conventions.py`. Convention drift in the code would break CI. In particular, the mixer stores its per-step eager term under a variant-agnostic key so a downstream probe cannot silently compute a different operator than the mixer used (an earlier revision of this line of work had exactly this failure mode).

A publisher-side parity script (not shipped with the reproduction repository) confirms the fresh implementation is byte-identical to the project’s rig-of-record reference implementations for both variants, and byte-identical to the reference task sampler across the K_1/K_2 configurations we train on.

4.6 Mechanism probes

We use three probes, all variant-aware and cache-safe:

- **F3 (logit lens)**. At each layer’s residual output, apply the model’s own RMSNorm and LM head; report the fraction of episodes whose ATOK residual decodes the answer token as the argmax restricted to the K_2 candidate value tokens.
- **F4 (mid-scan)**. At a chosen layer, cache (q, k, v, β) during the forward pass. Externally reconstruct the state M_t at every timestep using the mixer’s *own* eager operator method (so the probe cannot diverge from the mixer’s semantic). Compute $y'_t = q_{\text{ATOK}}^\top M_t$ at every t and fit a K_2 -way constrained logistic regression to $y'_t \rightarrow \{\text{bridge}, \text{answer}\}$. An optional knockout arm zeros the eager term during the reconstruction.
- **F5 (Q/K alignment)**. Three cosine measurements each with a same/control setup: (a) $\cos(q_{\text{ATOK}}, k[k_1^{\text{target}}])$, (b) $\cos(k[b_i], k[c_{f(i)}])$, (c) $\cos(v[b_i], k[e_i])$. Reports the mean over n batches.

5 Results

5.1 K-hop ladder

Table 1. Chain-composition accuracy on the K1 ladder under *mixed* training (4000 steps, $n = 10$ seeds for *rev*, $n = 5$ for *fwd*). Chance = 0.25. Numbers are per-seed final accuracies at each rung; passing seeds (final $K_1=4$ acc ≥ 0.85) are bolded. The one borderline seed (rev s8 at 0.895) sits closer to the passing cluster than the failing one.

Variant	Seed	$K_1 = 1$	$K_1 = 2$	$K_1 = 3$	$K_1 = 4$
rev	0	0.996	0.992	0.965	0.929
rev	1	0.994	0.977	0.955	0.914
rev	2	0.902	0.420	0.311	0.260
rev	3	0.970	0.503	0.335	0.257
rev	4	0.877	0.427	0.314	0.255
rev	5	0.898	0.460	0.310	0.262
rev	6	0.977	0.481	0.312	0.273
rev	7	0.995	0.979	0.962	0.942
rev	8	0.991	0.970	0.939	0.895
rev	9	0.992	0.975	0.944	0.917
fwd	0	0.996	0.476	0.329	0.265
fwd	1	0.761	0.508	0.376	0.328
fwd	2	0.819	0.401	0.305	0.253
fwd	3	0.998	0.493	0.330	0.273
fwd	4	0.991	0.492	0.342	0.263

Under mixed-mode training, the *rev* variant passes the full ladder to $K_1=4$ on 5/10 seeds at acc ≥ 0.895 (with the borderline s8 at 0.895 closer to the passing cluster than the failing one) and stalls at chance on the other 5/10. The *fwd* variant does not pass on any seed under this curriculum, plateauing at acc $\lesssim 0.33$ at $K_1 = 4$; we did not extend *fwd* to $n=10$ because the $n=5$ signal is already unambiguous.

Under joint-mode training, the *rev* variant passes the full ladder on 6/10 seeds at acc ≥ 0.958 , versus 5/10 (with one borderline at 0.895) under mixed-mode. Under both curricula the failing cluster lands cleanly at chance (acc ≤ 0.27), preserving a sharp bimodal formation pattern.

Curriculum rescues *fwd* formation partially. A notable finding under joint-mode is that the *fwd* variant, which showed 0/5 pass at $K_1 = 4$ under mixed-mode, has 1/5 pass under joint at acc = 0.977 (seed 3). This falsifies the reading that *fwd* fails categorically; the operator-direction asymmetry is better characterised as *fwd* having a much lower per-seed pass probability under matched protocol than *rev* (1/5 vs. 6/10 under joint), one that the joint curriculum partially rescues. The passing *fwd* trajectory crystallises later than passing *rev* seeds (rung $K_1=4$ reaches ≥ 0.9 near step 3200 for *fwd* s3 vs. steps 3000–4200 for the passing *rev* seeds).

5.2 Formation-mode bimodality

The pass-vs-fail split in Table 1 is not seed noise in the ordinary sense: passing seeds land at acc ≥ 0.91 at $K_1 = 4$, and failing seeds at acc ≤ 0.27 . The $K_1=2$ column also splits sharply between two clusters (≥ 0.98 vs $\lesssim 0.51$). This bimodal “formation” pattern is consistent with what earlier work on this mixer family reported anecdotally at $n=1-3$; the $n=5$ sample here quantifies the pass rate directly.

The training-time signature separates the two modes well before convergence. The first ladder rung (k_1)

Table 2. Same as Table 1 but under *joint* training (5000 steps, sampling one batch each at $K_1=1$ and $K_1=4$ per step, averaged CE). Joint mode dramatically improves the rev pass rate; fwd remains at chance.

Variant	Seed	$K_1 = 1$	$K_1 = 2$	$K_1 = 3$	$K_1 = 4$
rev	0	0.999	0.963	0.975	0.971
rev	1	0.999	0.977	0.979	0.983
rev	2	1.000	0.463	0.271	0.254
rev	3	0.999	0.988	0.988	0.984
rev	4	0.999	0.961	0.971	0.958
rev	5	0.998	0.418	0.287	0.250
rev	6	1.000	0.938	0.973	0.966
rev	7	1.000	0.946	0.988	0.979
rev	8	0.999	0.408	0.278	0.265
rev	9	0.999	0.421	0.290	0.265
fwd	0	0.999	0.457	0.280	0.250
fwd	1	0.999	0.453	0.278	0.262
fwd	2	0.999	0.413	0.268	0.256
fwd	3	0.999	0.975	0.978	0.977
fwd	4	0.999	0.408	0.279	0.249

saturates near 1.0 in essentially every run, passing and failing alike, so it carries no predictive signal on its own. The distinguishing event is the lift of the second rung: in passing seeds k_2 begins climbing above the $1/K_2$ floor partway through training and drags k_3 and k_4 up behind it, whereas failing seeds hold k_2 through k_4 at chance for the entire run. Under joint-mode training the passing seeds cross $\text{acc} = 0.95$ between roughly step 3000 and step 4400 (recorded per run as the `step_to_95` field in the committed result JSONs), while failing seeds never reach it. The fate of a run is thus legible from the k_2 trajectory long before the final step, but not from k_1 .

5.3 Mechanism localisation (F3)

A finding not present in the original single-seed reports of this mixer family is that passing seeds separate into two mechanism classes: half compose at the final layer (L_2 decodes strongly, L_1 does not) and half compose one layer earlier (L_1 already decodes, L_2 matches). Both classes achieve comparable $K_1=4$ accuracy. Whether the mixer forms its composition at the penultimate or final layer appears to be a seed-dependent choice uncorrelated with pass/fail. The one passing fwd seed also lands in the penultimate-layer class.

5.4 Mid-scan buildup (F4)

At L_1 on the passing rev s0 checkpoint, the answer signal $q_{\text{ATOK}}^\top M_t$ traces the following trajectory across positions (chance = 0.25):

position	...	$k_2[0]$	v_0	$k_2[1]$	v_1	$k_2[2]$	v_2	$k_2[3]$	v_3	ATOK
answer	...	0.27	0.27	0.33	0.32	0.47	0.47	0.56	0.64	0.82

The answer is at chance throughout the hop-1 write window, remains near chance early in the hop-2 window, and climbs monotonically through the last three value writes to 0.82 at ATOK. Bridge signal shows a symmetric pattern in the hop-1 window (details in the appendix).

Table 3. F3 logit-lens ATOK-decodable accuracy at each layer’s residual, chance = 0.25. Passing seeds show the answer becoming decodable at one of the final two residuals; failing seeds show no localisation anywhere. Bolded columns mark where each passing seed first exceeds 0.85. Note the mechanism-class split within passing seeds: some compose at L_1 (final layer), some already at L_2 (penultimate).

Variant	Seed (curriculum)	embed	after L_0	after L_1	after L_2
<i>passing seeds – compose at final layer L_2</i>					
rev	0 (mixed)	0.250	0.264	0.358	0.944
rev	1 (mixed)	0.271	0.259	0.259	0.926
rev	0 (joint)	0.249	0.323	0.326	0.973
rev	1 (joint)	0.271	0.275	0.257	0.988
rev	3 (joint)	0.256	0.258	0.254	0.990
rev	4 (joint)	0.238	0.237	0.271	0.962
<i>passing seeds – compose at penultimate layer L_1</i>					
rev	7 (mixed)	0.249	0.257	0.931	0.932
rev	8 (mixed)	0.250	0.453	0.898	0.902
rev	9 (mixed)	0.261	0.513	0.936	0.936
rev	6 (joint)	0.255	0.261	0.968	0.975
rev	7 (joint)	0.249	0.256	0.983	0.983
fwd	3 (joint)	0.256	0.249	0.981	0.981
<i>failing seeds – no localisation</i>					
rev	2 (joint)	0.260	0.253	0.252	0.246
rev	5 (joint)	0.257	0.246	0.255	0.252
fwd	0 (mixed)	0.252	0.250	0.249	0.262
fwd	0 (joint)	0.250	0.252	0.250	0.248
... (rest of failing seeds omitted for brevity, see <code>code/outputs/table2__*f3.json</code>)					

Causal test. Zeroing the eager term $a_t = M_{t-1}k_t$ during the reconstruction (F4 with `-eager-knockout`) collapses the ATOK answer signal on the same checkpoint to 0.267 (chance). The address-perturbation is causally necessary for the mid-scan buildup.

F4 sweep across representative checkpoints. Table 4 extends the F4 probe to 10 checkpoints at L_1 and adds an L_0 probe on the three passing seeds whose F3 decode only appeared at the final residual. Findings:

- **L_1 composers** (4 passing seeds): F4 shows large buildup (~ 0.94) at L_1 and full collapse to chance under KO. Matches the F3 pattern where the answer decodes at L_2 (the residual immediately after L_1 ’s mixer).
- **L_0 composer** (rev joint s0): F4 buildup shifts one block earlier – L_0 base = 0.965, L_0 KO = 0.280 – and L_1 shows no buildup. F3 decodes only at the final residual, consistent with the answer being computed early but requiring two more residuals to reach the LM head.
- **Diffuse composers** (rev joint s3, s4): Two passing seeds show no single-layer buildup at either L_0 or L_1 (both at chance), yet decode strongly at the final F3 layer and achieve $K_1=4 \geq 0.98$. The mechanism is presumably spread across L_2 or distributed across layers; we did not resolve which because the L_2 F4 probe exhausted its LR convergence budget on multiple attempts.
- **Failing seeds:** all show chance-level L_1 ATOK signal both with and without KO.

Table 4. F4 mid-scan ATOK answer signal, baseline (eager on) vs. *eager-KO* (eager term zeroed during reconstruction). Chance = 0.25. Passing seeds fall into three mechanism classes: L_1 *composers* show large buildup at L_1 ; L_0 *composers* show it at L_0 instead (with L_1 silent); *diffuse composers* show no single-layer buildup at either. KO collapses the buildup to chance in the classes where it exists.

Class	Variant/Seed	Probed	base	KO	Curriculum
<i>L_1 composers – F4 buildup at L_1; F3 decodes at L_2</i>					
	rev s0	L_1	0.917	0.267	mixed
	rev s7	L_1	0.935	0.280	mixed
	rev s6	L_1	0.958	0.262	joint
	fwd s3	L_1	0.986	0.277	joint
<i>L_0 composer – F4 buildup at L_0; F3 decodes only at final residual</i>					
	rev s0	L_0	0.965	0.280	joint
	rev s0	L_1	0.296	0.309	joint
<i>Diffuse composers – no single-layer buildup at L_0 or L_1; still pass</i>					
	rev s3	L_0	0.268	0.267	joint
	rev s3	L_1	0.281	0.273	joint
	rev s4	L_0	0.268	0.255	joint
<i>Failing seeds – chance at L_1 base and KO</i>					
	rev s2	L_1	0.258	0.273	joint
	rev s5	L_1	0.265	0.258	joint
	fwd s0	L_1	0.277	0.254	mixed
	fwd s0	L_1	0.262	0.256	joint

5.5 Q/K alignment (F5)

We ran the Q/K alignment probe on a small number of checkpoints rather than the full sweep, so we report it as preliminary and single-seed. On a passing rev seed (s0) at L_2 , prediction (b), $\cos(k[b_i], k[c_{f(i)}])$, shows a positive gap over its control (0.474 versus 0.436, a +0.038 margin); on a failing rev seed (s2) the same gap shrinks to +0.012. The gap is larger at L_1 , where the composition is being constructed rather than read out. These point measurements are directionally consistent with the F3 and F4 results (the aligned representation is present in passing seeds and absent in failing ones), but we draw no quantitative conclusion from two seeds; a systematic multi-seed sweep is left as future work.

6 Ablations and analysis

6.1 Inference-time knockout on trained checkpoints

The F4 knockout in §5.4 shows that zeroing the eager term $a_t = M_{t-1}k_t$ during the *probe*’s state reconstruction collapses the answer signal to chance. A stronger inference-time test replaces the mixer’s own forward-pass eager term with zero on trained weights and evaluates the model on the actual task. If the model has wound the eager term into its computation, the prediction accuracy will not simply drop to chance (0.25); it will be systematically *wrong* because the weights are calibrated around the perturbation.

The KO accuracy for passing rev seeds is not just below chance — it is an order of magnitude below — meaning the model does not degrade to a random-token prior when the eager term is removed; it emits confidently wrong tokens. The eager perturbation is the computational pathway the model routes through, not a small correction term. The fwd collapse to $\lesssim 0.001$ hits (both for the one passing seed and for every

Table 5. Inference-time eager-KO on trained checkpoints, grouped by seed category. Same model weights evaluated with the eager term forced to zero in the mixer’s forward pass. Chance = 0.25. Passing **rev** seeds collapse by $\sim 30\times$ to well below chance; failing **rev** seeds show smaller but real degradation; **fwd** seeds (passing and failing) collapse to $\lesssim 0.001$ hits under KO, exceeding the **rev** collapse. Group means over n seeds; per-seed data committed alongside.

Group	n	Setting	$K_1=1$	$K_1=2$	$K_1=3$	$K_1=4$
rev passing	11	eager on	0.996	0.969	0.967	0.952
		eager off	0.042	0.038	0.034	0.032
rev failing	9	eager on	0.957	0.464	0.309	0.260
		eager off	0.092	0.074	0.065	0.059
fwd passing	1	eager on	0.999	0.968	0.980	0.981
		eager off	0.000	0.000	0.000	0.000
fwd failing	9	eager on	0.950	0.478	0.322	0.269
		eager off	0.016	0.005	0.002	0.000

failing seed) is stronger still: even the small hop-1 competence the failing **fwd** baselines display ($k_1 \sim 0.94$) is downstream of the eager term, and without it the model produces essentially no correct answer at any rung.

6.2 Training-time causal control: eager term removed at training

The inference-time knockout above shows that trained weights depend critically on the eager term at inference. A separate question is whether the mechanism can *form at all* without the eager term. We train a paired ablation under matched protocol (joint mode, 5000 steps, $n = 5$ seeds; **rev** and **fwd** reduce to the same model when the eager term is removed, and per-seed training was byte-identical, see caption below) with the eager term disabled at every step ($k_{\text{eff}} = k$, standard fast-weight write with no address perturbation).

Table 6. Ablation: eager term removed at training time (joint curriculum, $n = 5$ seeds). Compare with Table 2 (same curriculum, eager term on). Without the eager term the **rev** and **fwd** variants collapse to the same model (both reduce to $k_{\text{eff}} = k$, a standard fast-weight write); we verified this by running both — final accuracies are byte-identical per seed. $K_1=1$ single-hop retrieval works perfectly on every seed (the outer-product write suffices), but chain composition to $K_1=4$ plateaus far below the eager-on ceiling: one of five seeds reaches $\text{acc} = 0.503$ ($\sim 2\times$ chance), the other four stall at chance. The eager address-perturbation is what lifts chain composition from ~ 0.50 (partial residual-based pathway) to ~ 0.97 (eager-mediated pathway).

Seed	$K_1 = 1$	$K_1 = 2$	$K_1 = 3$	$K_1 = 4$
0	1.000	0.660	0.566	0.503
1	1.000	0.507	0.303	0.261
2	1.000	0.472	0.266	0.272
3	1.000	0.479	0.276	0.282
4	1.000	0.422	0.273	0.255

Mechanism of the partial-pass ablated seed

Probing the one partial-passing ablated seed (s_0 , $k_4=0.503$) with F3 and F4 reveals a mechanism qualitatively different from the eager-on passing seeds of §5.3:

- **F3 (logit lens).** $L_0=0.250$, $L_1=0.249$, $L_2=0.470$, $L_3=0.492$. Only the final two residuals lift above chance, and only to ~ 0.49 , matching the seed’s k_4 accuracy.

- **F4 (mid-scan buildup, ATOK answer signal).** L=0 ATOK= 0.268; L=1 ATOK= 0.379; L=2 ATOK= 0.211. Weak signal at L=1, no clean per-layer buildup. The eager-mediated pathway that gives the sharp L=0-or-L=1 buildup in the eager-on passing seeds is absent.

The residual-based pathway that survives without the eager term is weaker and more diffuse. It reaches ~ 0.5 where the eager-mediated pathway reaches ~ 0.97 , and it does not localise to a single mixer’s write. This is consistent with the interpretation that the eager term is not a small booster but the primary computational channel for single-pass chain composition on this task.

6.3 Other planned ablations (future work)

- vs standard DeltaNet ($k, v - M^\top k$) as a canonical delta-family baseline
- vs Mamba2 as a non-fast-weight strong SSM baseline

7 Related work

The lineage. The primitive we study sits inside the fast-weight programmer family, which runs from Schmidhuber (1992) through Schlag et al. (2021); Irie et al. (2022b) to the recent DeltaNet work of Yang et al. (2024b,a). All of these share the outer-product write $M += \beta_t k'_t v'^\top_t$; they differ in how k'_t and v'_t depend on the current state. Delta-family mixers modify the *value* being written (the residual $v_t - M^\top k_t$); FENRIR modifies the *key* of the write via $k_{\text{eff},t} = k_t + M_{t-1} k_t$ (reverse-lookup variant) or $k_t + M_{t-1}^\top k_t$ (forward-lookup variant).

On novelty of the primitive. We do not claim strong novelty for the FENRIR update rule. Prior work on fast-weight programmers, self-referential weight matrices (Irie et al., 2022b), and state-augmented linear-attention variants has explored a large space of state-dependent write dynamics, and it is plausible that an update functionally equivalent to FENRIR has appeared under another name. We searched the literature we know (§7.1 below) and did not find an exact match; we welcome corrections. What we claim in this paper is not the primitive but a mechanistic account of how one instance of this family learns (and often fails to learn) 2-hop chain composition inside a single forward pass.

7.1 Closest prior art we found

The DeltaNet family (Irie et al., 2022b; Yang et al., 2024b,a) is structurally closest: same outer-product write, same forward read, same $O(d^2)$ state, similar chunk-parallel kernel for the reverse-lookup variant. The intervention is in a different place (value vs key). GatedDeltaNet (Yang et al., 2024a) adds scalar gating on the residual, which is compatible with FENRIR’s address perturbation in principle.

We searched several other lines of fast-weight and linear-attention work for a state-dependent modification to the write key before the outer product. The closest analogs we found each apply their state-dependent term in a different place than FENRIR does:

- Ba et al. (2016) iterate a fast-weight read $h^{s+1} = \text{LN}(Wh_{\text{prev}} + Cx + Ah^s)$ inside the recurrent step: the fast-weight matrix A is read with the current hidden state and the result is added back into that state. The state-dependent additive is structurally similar to FENRIR’s but strictly on the READ side; A is not built afterwards from the modified vector.
- Le et al. (2026) apply $q_{\text{clean}} = (I - \lambda \Sigma)q$ where Σ is a running covariance of past keys, a state-dependent additive correction to the QUERY on the READ side. Mirror image of FENRIR’s intervention, not the same one.
- Park et al. (2026) build a mixed vector $k + \lambda q$ that appears in the erase term $(I - \beta(kk^\top + \lambda qk^\top))$; still

not a state-dependent addition to k before the write.

- Self-referential weight matrices (Irie et al., 2022a) let a fast-weight matrix generate its own q, k, v, β projections from the current state, then apply a delta update. Self-reference is on GENERATION of the projections, not on a $k + Mk$ residual after projection.

We surveyed 15+ additional variants (DeltaProduct (Siems et al., 2025), MesaNet, Longhorn, Titans, TTT-Linear (Sun et al., 2024), Kaczmarz Linear Attention, and the wider recurrent-linear-attention family) and did not find a closer match.

The two closest self-referential fast-weight families were verified against FENRIR’s update rule directly:

- Recurrent Fast Weight Programmers (Irie et al., 2021) include a Recurrent Delta Net (RDN) whose k, v, q, β projections are augmented with a state-dependent term $R_\bullet \tanh(y^{(i-1)})$ derived from the previous output. This is state-dependent only through the previous output vector $y^{(i-1)}$ (a d -dim quantity routed through additional slow weights); no term of the form Mk enters the write. Not the FENRIR update.
- The Modern Self-Referential Weight Matrix (Irie et al., 2022a) generates its own projections $k_t = [W_{t-1}]_k \phi(x_t)$ from the current fast-weight matrix (i.e. the state IS the projection matrix), then applies a standard delta write $W_t = W_{t-1} + \sigma(\beta_t)(v_t - \bar{v}_t) \otimes \phi(k_t)$. FENRIR uses a fixed slow-weight projection $k_t = W_k x_t$ and adds an explicit state-dependent perturbation $k_{\text{eff}} = k_t + M_{t-1} k_t$ before the write. Different level of self-reference; not the same update.

We were unable to find a match for FENRIR’s specific update rule in the fast-weight or linear-attention literature we surveyed. We do not claim exhaustive coverage of obscure workshops, non-arXiv venues, or in-progress preprints, and welcome citations we missed. The paper’s argument (formation-mode bimodality, mid-scan mechanism, causal knockout) does not rest on primitive novelty.

7.2 Chain composition and mechanism study

Composition benchmarks. Grokked-Transformers (Wang et al., 2024) argues dense transformers reach reliable multi-hop composition only after extended training and frames the capability as a target for architectural inductive biases. Zoology (Arora et al., 2023) and BASED (Arora et al., 2024) isolate single-hop associative recall as a linear-attention weak spot; they do not probe multi-hop composition inside a single forward pass. Our task (injective 2-hop chains with truncated supervision) is intentionally minimal: it forces the model to link two facts through a shared bridge in one pass, with a clean $1/K_2$ chance floor.

Mechanism probes. Logit-lens (nostalgebraist, 2020) inspired our F3 layer localisation probe. Induction-head circuit analysis (Olsson et al., 2022) inspired the paired-knockout pattern we use in F4. Both techniques were originally applied to attention residual streams; we adapt them to the fast-weight state directly. Formation-mode bimodality is a phenomenon adjacent to grokking (Wang et al., 2024) in the sense that a fraction of training runs pass a capability transition and the rest do not; we do not attempt to bridge the two literatures here.

State-space baselines. Mamba and Mamba2 (Gu and Dao, 2023; Dao and Gu, 2024) do not carry a fast-weight state and are not intended to compete on tasks that require in-context storage-and-retrieval. We treat Mamba2 as a swap-in architectural baseline in the planned ablations, not as a competitor.

8 Limitations

- **Formation bimodality.** At $n = 5$ seeds under mixed training the rev variant passes 2/5. We do not have a training-time diagnostic that predicts which seeds will pass, and we do not offer a recipe that raises the pass rate.

- **Task scope.** Every claim in this paper is about the 2-hop injective truncated composition task at $K_1 \in \{1, \dots, 4\}$, $K_2 = 4$, at $d_{\text{model}} = 256$, $d_{\text{key}} = 32$, 3 FENRIR blocks, on synthetic data. Whether the mechanism survives naturalistic data or deeper chains is out of scope.
- **Variant coverage.** We report two operator directions (rev, fwd) and two training curricula (mixed, joint). Other axes – optimizer sensitivity, alternative scaffolds, learned α -gating on the eager term – are not ablated.
- **No LM transfer here.** The paper argues about a composition mechanism on a controlled task; it does not demonstrate that FENRIR is a competitive language-model mixer at any scale.

9 Conclusion

Chain composition inside a single mixer forward pass is attainable but formation-mode contingent: on the address-perturbed fast-weight mixer we study, a fraction of seeds train to full $K_1=4$ accuracy and the rest stall at chance. The passing seeds carry a mid-scan mechanism signature that a knockout of the address-perturbation term causally destroys. The reproduction code and audit workflow are packaged so future extensions can be checked against the convention tests before any new claim leaves the repository.

References

- Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, and Christopher Ré. Zoology: Measuring and improving recall in efficient language models. *arXiv preprint arXiv:2312.04927*, 2023.
- Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalsina, Silas Alberti, Dylan Zinsley, James Zou, Atri Rudra, and Christopher Ré. Simple linear attention language models balance the recall-throughput tradeoff. In *International Conference on Machine Learning (ICML)*, 2024.
- Jimmy Ba, Geoffrey Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- Tri Dao and Albert Gu. Transformers are SSMS: Generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- Kazuki Irie, Imanol Schlag, Róbert Csordás, and Jürgen Schmidhuber. Going beyond linear transformers with recurrent fast weight programmers. *arXiv preprint arXiv:2106.06295*, 2021.
- Kazuki Irie, Imanol Schlag, Róbert Csordás, and Jürgen Schmidhuber. A modern self-referential weight matrix that learns to modify itself. *arXiv preprint arXiv:2202.05780*, 2022a.
- Kazuki Irie, Imanol Schlag, Róbert Csordás, and Jürgen Schmidhuber. A modern self-referential weight matrix that learns to modify itself. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022b. Contains the delta-rule fast-weight variant referred to as DeltaNet in later work.
- Dong Le, Thong Nguyen, Cong-Duy Nguyen, and Anh Tuan Luu. Don’t read everything: A curvature-conditioned query for linear attention. *arXiv preprint arXiv:2606.01294*, 2026.
- nostalgebraist. interpreting GPT: the logit lens. <https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>, 2020. LessWrong post.

- Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, et al. In-context learning and induction heads. *Transformer Circuits Thread*, 2022. URL <https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/>.
- Sumin Park, Seojin Kim, and Noseong Park. Q-Delta: Beyond key-value associative state evolution. *arXiv preprint arXiv:2606.08804*, 2026.
- Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.
- Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.
- Julien Siems, Timur Carstensen, Arber Zela, Frank Hutter, Massimiliano Pontil, and Riccardo Grazi. DeltaProduct: Improving state-tracking in linear RNNs via householder products. *arXiv preprint arXiv:2502.10297*, 2025.
- Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024.
- Boshi Wang, Xiang Yue, Yu Su, and Huan Sun. Grokked transformers are implicit reasoners: A mechanistic journey to the edge of generalization. *arXiv preprint arXiv:2405.15071*, 2024.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024a.
- Songlin Yang, Bailin Wang, Yikang Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2024b.

A Chunk-parallel kernel derivation (rev variant)

The rev recurrence at position t is

$$a_t = M_{t-1}k_t, \quad w_t = \beta_t(k_t + a_t), \quad (5)$$

$$M_t = M_{t-1} + w_t v_t^\top, \quad y_t = q_t^\top M_t. \quad (6)$$

Fix a chunk of size c with entry state M_0 . For a position t inside the chunk, $M_{t-1} = M_0 + \sum_{s < t} w_s v_s^\top$, so

$$w_t = \beta_t \left(k_t + M_0 k_t + \sum_{s < t} (v_s^\top k_t) w_s \right). \quad (7)$$

Stack the rows $w_t^\top$ into $W \in \mathbb{R}^{c \times d}$ and the keys $k_t^\top$ into $K \in \mathbb{R}^{c \times d}$. The coupled equations above become a single linear system,

$$(I - \text{diag}(\beta) T) W = \text{diag}(\beta) (K + P), \quad P := K M_0^\top, \quad T_{ts} := v_s^\top k_t \quad (s < t), \quad (8)$$

where T is strictly lower triangular, so $I - \text{diag}(\beta) T$ is unit lower triangular. W is therefore recovered by one batched triangular solve per chunk (`torch.linalg.solve_triangular`), with no per-position Python loop. The chunk output and exit state then follow as

$$Y = Q M_0 + \text{tril}(Q W^\top) V, \quad M_c = M_0 + W^\top V, \quad (9)$$

where the tril includes the diagonal (read-after-write). The per-chunk cost is $O(d^2 c + d c^2 + c^3)$ and the serial depth is L/c instead of L .

Only the `rev` variant admits this triangular closure. The `fwd` eager term $M^\top k$ produces a bilinear reduction over the past, $\sum_{s < t} (w_s^\top k_t) v_s$, that does not collapse to a single triangular solve without further algebra, so the `fwd` variant falls back to the sequential path in `model.py`; a chunk-parallel `fwd` form is left as an engineering follow-up. The state M is accumulated in `fp32` regardless of the caller’s dtype, because `bf16` accumulation of this long-horizon sum degrades retrieval; a chunked-versus-sequential parity test (`tests/test_parity.py`) pins the two paths to the same output.

B Audit workflow and the operator-mismatch bug class

The mechanism probes (Section 4.6) reconstruct the mixer state M_t outside the module so that individual terms can be knocked out. That reconstruction is only trustworthy if it uses the *same* operator the mixer used. The failure mode we guard against is an external einsum whose contraction axis does not match the mixer’s storage convention: under the convention $M[i, j] = k_{\text{eff}}[i] v[j]$, the two eager terms Mk and $M^\top k$ differ only by which axis is contracted, so a probe that picks the wrong axis runs without error and returns plausible numbers while silently measuring a different operator than the model was trained with. On this project that class of bug was caught twice before it reached a reported number.

The safeguard we adopted has two parts. First, the mixer exposes its per-step eager term under a variant-agnostic attribute, so a probe reads the mixer’s own operator rather than recomputing one and risking a mismatch. Second, every storage-convention and per-variant operator claim is pinned as a `pytest` assertion in `code/tests/test_conventions.py`, so a convention change breaks CI rather than a result. The passing test suite is the audit. The internal audit document that originally motivated this discipline is publisher-internal and is not shipped with the reproduction repository.